

tmux documentation

§04 — SSH in Termux

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** Termux's OpenSSH and how it interacts with v1.0.9's `tmx-ssh-dispatch.sh`

1. Termux ships standard OpenSSH

`pkg install openssh` installs a regular OpenSSH build, currently **10.3p1** per the package recipe at <https://github.com/termux/termux-packages/blob/master/packages/openssh/build.sh>. This is the same upstream OpenSSH used on Linux and macOS, with Termux-specific path adjustments.

Build-time customisations (per the recipe):

- `--sysconfdir=$TERMUX_PREFIX/etc/ssh` — `sshd_config` lives at `/data/data/com.termux/files/usr/etc/ssh/sshd_config`
- `--with-default-path=$TERMUX_PREFIX/bin` — `sshd`-launched processes get Termux's `PATH`
- `--with-pid-dir=$TERMUX_PREFIX/var/run` — PID file lives under the app's data dir
- `--without-ssh1` — protocol 1 disabled (good; it's deprecated everywhere)
- `utmp/wtmp/lastlog` disabled — Android has no shared `utmp` infrastructure

Crucially, **the** `command=` **directive in** `authorized_keys`, **the** `Match/ForceCommand` **keywords in** `sshd_config`, **and the** `SSH_ORIGINAL_COMMAND` **env var all work identically to upstream OpenSSH** — they are core OpenSSH features and Termux's build does not touch them.

2. Default port: 8022

Ports 1-1024 are privileged on Linux (bind requires `CAP_NET_BIND_SERVICE` or root). Termux runs as an unprivileged Android app and therefore cannot bind to port 22. Default is **port 8022**.

The default `sshd_config` that Termux ships sets Port 8022 explicitly. Source: confirmed in numerous Termux setup guides — https://jonesbalada.gitlab.io/18-dezoito/2018-12-05-run_ssh_termux.html, <https://termuxtools.com/ssh-server-termux-setup-guide-2/>.

Client incantation:

```
ssh -p 8022 user@phone.local      # local network
ssh -p 8022 user@1.2.3.4          # if exposed via static IP
ssh nezha-tmx work                # using a ~/.ssh/config Host alias (preferred)
```

3. Username

Termux assigns the operator a username derived from the Android UID (u0_a274 shape). For the `command= dispatch` design, the username doesn't matter — what matters is the SSH key, not the local identity. But the `ssh user@host` syntax requires SOME name; the standard incantation in the Termux community is `ssh -p 8022 $(whoami)@hostname` after first running `whoami` once to discover the actual UID-derived name.

Operators often add a `User u0_a274` line to the client-side `~/.ssh/config` Host block. Our `tmx-ssh-install.sh` should detect the remote username automatically via `ssh remote-host whoami` during initial install and bake it into the Host block.

4. `command= directive` — does it work?

Yes. OpenSSH 10.3p1's `command= directive` is a server-side enforcement: when the matching public key authenticates, `sshd` unconditionally `exec's` the specified command instead of the user's shell. The user's actual `$SHELL` does not run. The user-supplied command (if any) is exposed in `$SSH_ORIGINAL_COMMAND`.

`authorized_keys` format works the same:

```
command="$PREFIX/bin/tmx-ssh-dispatch.sh",no-port-forwarding,no-X11-forwarding
```

(Note: the `no-pty` restriction would BREAK us because we WANT a PTY for tmux. Omit `no-pty` from the directive list.)

Source for `command=` mechanics: https://manpages.debian.org/experimental/openssh-server/authorized_keys.5.en.html, and the Linux-side man page applies identically.

5. `$SSH_ORIGINAL_COMMAND` propagation

When the client runs `ssh nezha-tmx work`, `sshd` authenticates the key, sees the `command= directive`, and `exec's` `tmx-ssh-dispatch.sh` with these env vars set:

- `SSH_ORIGINAL_COMMAND=work` (the user-supplied command)
- `SSH_CONNECTION=<client-ip> <client-port> <server-ip> <server-port>`
- `SSH_CLIENT=<client-ip> <client-port> <server-port>`
- `SSH_TTY=/dev/pts/N` (if PTY allocated)
- `LANG`, `TERM`, `PATH`, etc. inherited from `sshd`'s environment

The `command= path` takes priority over the user's shell — the dispatcher script does NOT have to source `.bashrc`. It MUST set up its own `PATH` if it needs more than `$PREFIX/bin`. (Termux's default `PATH` is just `$PREFIX/bin`, which is fine for our purposes — `tmx` and `tmux` both live there.)

6. `$PATH` propagation — confirmation

The `sshd` build flag `--with-default-path=$TERMUX_PREFIX/bin` ensures every `sshd`-spawned process gets `$PATH=/data/data/com.termux/files/usr/bin`. This is exactly what the dispatcher needs: it can call `tmx attach -t NAME` directly without absolute paths.

If we want to be defensive, the dispatcher prologue should do:

```
PATH="${PREFIX:-/data/data/com.termux/files/usr}/bin:$PATH"
export PATH
```

This handles the edge case of a misconfigured sshd or a custom Termux PREFIX.

7. tmx-ssh-dispatch.sh portability check

Walking the spec §5.2 logic against Termux:

```
case "${SSH_ORIGINAL_COMMAND:-}" in
    "")
        exec bash -l ;;
        #
        interactive — works on Termux
    '*' | '*' ';' *) printf 'reject\n' >&2; exit 1 ;;    # POSIX
        case patterns — works
    *)
        case "$SSH_ORIGINAL_COMMAND" in
            [A-Za-z0-9_.-]*)
                exec tmx attach -t "$SSH_ORIGINAL_COMMAND" ||
                    exec tmx new -s "$SSH_ORIGINAL_COMMAND"
                ;;
            *) printf 'reject\n' >&2; exit 1 ;;
        esac
    ;;
esac
```

POSIX-sh patterns are bash-3.2-and-up compatible (§11.4.67 mandate). Termux ships bash 5.x so we are well-supported. `exec bash -l` works because bash is at `$PREFIX/bin/bash`.

One thing to verify on Termux specifically: does `exec bash -l` properly source the Termux login chain? Termux's bash login chain is: `/data/data/com.termux/files/usr/etc/bash.bashrc` (system-wide) → `~/.bash_profile` → `~/.profile` → `~/.bashrc`. UNCONFIRMED: I have not confirmed bash -l follows this exact chain inside Termux; some Termux community guides claim Termux's bash was patched to merge `.bash_profile` and `.bashrc` semantics, but the patch is not documented in the upstream package recipe.

Resolution path: run `bash -lc 'echo $TERMUX_VERSION'` interactively after `ssh nezha-tmx` — if `$TERMUX_VERSION` is set, the login chain ran; if empty, we have to source `$PREFIX/etc/bash.bashrc` explicitly in the dispatcher.

8. Host alias on the client

The v1.0.9 design writes a Host block to `~/.ssh/config`:

```
Host nezha-tmx
    HostName nezha.local
    Port 22
    User milosvasic
    IdentityFile ~/.ssh/id_tmx_nezha
    IdentitiesOnly yes
```

For a Termux target the block needs Port 8022:

```
Host phone-tmx
  HostName phone.local
  Port 8022
  User u0_a274
  IdentityFile ~/.ssh/id_tmx_phone
  IdentitiesOnly yes
```

The `tmx-ssh-install.sh` installer should detect the remote `sshd` port. Today's spec hardcodes 22; we'd add a `--port 8022` flag, or auto-probe by trying `ssh -p 8022 hostname true && PORT=8022` before falling back.

9. authorized_keys path on Termux

Standard: `$HOME/.ssh/authorized_keys` → `/data/data/com.termux/files/home/.ssh/authorized_keys`. Mode 0600. `.ssh/` directory mode 0700. OpenSSH refuses to use them if perms are too open — same as anywhere else.

The Termux package install creates `$HOME/.ssh/` with the right perms IF the operator runs `ssh-keygen` once. Our installer should be defensive and `mkdir -p -m 0700 ~/.ssh && chmod 0700 ~/.ssh && touch ~/.ssh/authorized_keys && chmod 0600 ~/.ssh/authorized_keys`.

10. Starting sshd

Termux's `sshd` does NOT auto-start. The operator runs `sshd` once (which forks the daemon — `sshd` defaults to daemon-mode). Source: <https://gist.github.com/devmaars/8e33a1edefc4b048a433651a1fc89844>.

For persistent `sshd` across phone reboots: install Termux:Boot (f-droid.org/en/packages/com.termux.boot/) and place a script at `~/.termux/boot/start-sshd`:

```
#!/data/data/com.termux/files/usr/bin/sh
termux-wake-lock
sshd
```

Source: <https://github.com/termux/termux-boot> + the Termux:Boot wiki.

This is the canonical pattern. Our installer should add this script when the operator opts into autostart.

11. Security posture

- The `command=restricted` key authorises ONLY session attaches. Cannot run arbitrary commands, cannot `scp/sftp` (`sftp` goes through a different subsystem handled by `sshd` before `command=` fires — but `command=` overrides it).
- The operator's normal shell-access key remains separate, so administrative access doesn't go through the restricted dispatcher.
- Adding `no-port-forwarding,no-X11-forwarding,no-agent-forwarding` to the directive line is recommended defence-in-depth. (Keep `no-pty` OUT — we need the PTY.)

UNCONFIRMED: whether Termux's sshd respects all the no - * options as a regular OpenSSH would. The build doesn't patch the relevant code paths so I have high confidence they work, but a runtime smoke test should explicitly verify each restriction. **Resolution path:** Test 22 in spec §7.3 expanded with one assertion per restriction.

12. Practical end-to-end demo (target state after Termux port)

```
# On the phone — first-time setup:
pkg install openssh tmux git
ssh-keygen -t ed25519                                # creates ~/.ssh/
                                                       id_ed25519
# Add the operator's PUBLIC key (from laptop) to ~/.ssh/
  authorized_keys with command= directive
sshd                                                  # start daemon
termux-wake-lock                                     # keep alive in
  background

# On the laptop — first-time setup:
ssh-keygen -t ed25519 -f ~/.ssh/id_tmx_phone
# Add Host block to ~/.ssh/config with Port 8022
# Copy public key to phone's authorized_keys

# Daily use, laptop side:
ssh phone-tmx work
# → laptop connects to phone:8022 → sshd authenticates → command=
  fires → tmx-ssh-dispatch
# → exec tmx attach -t work || exec tmx new -s work
# → operator is inside session `work` on the phone
```

End user experience matches the macOS/nezha experience modulo the Port 8022 + the termux-wake-lock recommendation.

Sources

- <https://github.com/termux/termux-packages/blob/master/packages/openssh/build.sh> — Termux's OpenSSH recipe
- https://manpages.debian.org/experimental/openssh-server/authorized_keys.5.en.html — authorized_keys command= semantics
- https://jonesbalada.gitlab.io/18-dezoito/2018-12-05-run_ssh_termux.html — practical Termux SSH setup guide
- <https://termuxtools.com/ssh-server-termux-setup-guide-2/> — port 8022 default confirmation
- <https://gist.github.com/devmaars/8e33a1edefc4b048a433651a1fc89844> — sshd start procedure on Termux
- <https://github.com/termux/termux-boot> — Termux:Boot for autostart
- <https://github.com/termux/termux-packages/discussions/18318> — community discussion of sshd accessibility